

SGIM

Systeme de Gestion des Incidents Maritimes

Documentation technique du Backend Django

MRCC Abidjan • Centre secondaire MRSC San Pedro

Version du cahier des charges : v2 (Spécifications de l'Interface et Organisation des Quarts)

Ce document explique, de manière pédagogique, l'architecture du backend construit pour la plateforme SGIM : les choix techniques, la structure du projet, le fonctionnement du module central « Alerte », le système de rôles et de permissions, ainsi que les tests réalisés pour valider le bon fonctionnement de l'ensemble.

Document préparé pour l'équipe de développement SGIM.

Stack : Python / Django / Django REST Framework / MySQL / JWT

1. Contexte et objectifs du projet

Le MRCC (Maritime Rescue Coordination Center) d'Abidjan coordonne les opérations de recherche et de sauvetage en mer, avec le MRSC (Maritime Rescue Sub-Center) de San Pedro comme centre secondaire. Avant ce projet, les données d'incidents étaient saisies de façon disparate, ce qui nuisait à la traçabilité et à la coordination.

L'objectif du SGIM est de centraliser toute cette activité dans une plateforme numérique unique, avec un tableau de bord de pilotage consultable en temps réel par la Direction Générale et le Ministère des Transports.

Évolution majeure du cahier des charges (version 2)

En cours de développement, le cahier des charges a été révisé en profondeur. Les changements les plus structurants pour le backend ont été :

- **Comptes génériques par quart** : au lieu de ~100 comptes individuels, le centre fonctionne avec des comptes partagés par équipe (Car 1 à Car 4 à Abidjan, MRSC à San Pedro), plus des comptes individuels Administrateur / Super Administrateur.
- **Fusion Alerte / Incident** : les deux notions distinctes (une alerte qu'on « qualifie » en incident) ont été fusionnées en une seule entité, plus simple à gérer et plus fidèle au terrain.
- **Catégories d'alertes simplifiées** : passage d'une quinzaine de types détaillés à quatre grandes catégories opérationnelles (Sûreté, Pollution Maritime, Urgences Médicales en Mer, Sécurité Environnementale).
- **Traçabilité par signature manuelle** : chaque saisie doit porter le nom de famille de l'opérateur qui l'a réellement traitée, en plus du compte de connexion partagé.
- **Deux nouveaux modules** : le Rapport de fin de journée (envoyé automatiquement par email à la hiérarchie) et le module Réunions (comptes-rendus avec pièce jointe PDF).

2. Architecture technique

Le backend est une API REST construite avec Django et Django REST Framework (DRF). Il ne produit aucune page web à afficher : il expose des données au format JSON, que le frontend (développé séparément) consomme pour construire l'interface visible par les utilisateurs.

Composant	Choix technique	Rôle
Langage / Framework	Python 3 + Django	Structure du projet, ORM (accès à la base de données sans écrire de SQL), interface d'administration intégrée
API	Django REST Framework	Transforme les modèles Django en endpoints JSON consommables par le frontend
Authentification	JWT (JSON Web Token)	Chaque utilisateur reçoit un « jeton » temporaire après connexion, à présenter à chaque requête
Base de données	MySQL (via XAMPP en développement)	Stockage persistant de toutes les données métier
CORS	django-cors-headers	Autorise le frontend (hébergé sur un autre domaine/port) à appeler l'API
Filtrage / recherche	django-filter	Permet de filtrer les listes (ex : alertes par statut) directement via l'URL

Pourquoi JWT plutôt qu'une authentification classique par session ? Le frontend et le backend sont deux projets séparés, potentiellement hébergés sur des serveurs différents. Le JWT est un standard indépendant du navigateur : il fonctionne aussi bien depuis une application web que depuis une future application mobile.

3. Structure du projet : les applications Django

Django organise le code en « applications » (apps) : des modules indépendants, chacun responsable d'une partie du métier. Voici les onze applications qui composent le backend SGIM :

Application	Rôle métier	Accès API
accounts	Comptes utilisateurs, rôles (Super Admin / Admin / Opérateur) et équipes de quart	/api/users/
centers	Les deux centres : MRCC Abidjan et MRSC San Pedro	/api/centers/
references	Toutes les listes déroulantes de l'application (sources d'alerte, priorités, catégories...)	/api/references/...
vessels	Fiches navires (nom, type, pavillon, MMSI, IMO...)	/api/vessels/
partners	Partenaires externes (Marine Nationale, CIAPOL, Clinique Farah...)	/api/partners/ (Admin uniquement)
alerts	Cœur du système : l'Alerte, fusionnée avec l'ancien concept d'Incident	/api/alerts/
persons	Personnes et victimes rattachées à une alerte	/api/persons/
sar	Moyens de secours (vedettes, hélicoptères...) et leurs engagements sur une alerte	/api/sar/
event_logs	Journal des événements, alimenté automatiquement	/api/event-logs/ (Admin uniquement)
reports	Rapport de fin de journée et tableau de bord (KPI)	/api/reports/...
meetings	Comptes-rendus de réunion avec pièce jointe PDF	/api/meetings/

4. Le bug des listes déroulantes : diagnostic et correction

Le cahier des charges initial signalait un problème récurrent : certaines sélections faites dans les listes déroulantes du formulaire « Nouvelle Alerte » n'étaient pas correctement enregistrées.

Cause identifiée

En analysant la base Excel fournie, on a constaté que les libellés des référentiels (types de navires, types d'incidents...) contenaient des espaces incohérents : par exemple " Pirateries" au lieu de "Pirateries", ou "Porte-conteneurs " avec des espaces en trop à la fin. Quand ces libellés étaient utilisés comme valeur de sélection en texte brut, une valeur envoyée par le formulaire ne correspondait plus exactement à l'entrée attendue côté serveur — la sélection semblait donc « ignorée ».

Solution mise en place

- Chaque référentiel (source d'alerte, catégorie, type de navire, etc.) est désormais une vraie table en base de données, avec un identifiant unique et stable (UUID).
- Le frontend n'envoie plus jamais de texte pour une liste déroulante : il envoie l'identifiant choisi, obtenu au préalable via l'API (ex : GET /api/references/incident-categories/).
- À l'enregistrement, chaque libellé est automatiquement « nettoyé » (suppression des espaces en trop, en début, en fin, et entre les mots) avant d'être sauvegardé, avec une contrainte d'unicité sur cette valeur nettoyée — il devient donc impossible de recréer un doublon à cause d'un espace.

Résultat concret : une fois qu'un identifiant est choisi côté frontend, il ne peut plus jamais se « perdre » à cause d'une différence de mise en forme du texte — le problème est réglé à la racine.

5. Comptes, rôles et permissions

Le système distingue le niveau d'accès (ce qu'un compte a le droit de faire) de l'équipe d'appartenance (utile uniquement pour les comptes Opérateur).

Rôle	Description	Accès particulier
Super Administrateur	Contrôle total, y compris gestion des comptes utilisateurs	Toutes les routes, y compris /api/users/
Administrateur	Direction / support technique : vue consolidée, historique, validation des données	Partenaires, Journal des événements
Opérateur (Car 1-4 / MRSC)	Saisie des alertes, consultation cartographique, rapports de fin de journée	Alertes, navires, moyens, rapports — pas les Partenaires ni le Journal

Techniquement, chaque compte Opérateur porte aussi un champ « équipe » (Car 1, Car 2, Car 3, Car 4 ou MRSC), utilisé pour pré-remplir automatiquement le centre d'appartenance lors de la création d'une alerte — l'opérateur n'a donc rien à choisir manuellement à ce sujet.

Séparément, chaque alerte porte un champ `operator_signature` : le nom de famille tapé manuellement par l'opérateur qui a traité l'événement, exigé par le cahier des charges pour la traçabilité individuelle malgré l'usage de comptes partagés.

6. Cycle de vie d'une alerte

Le modèle `Alert` est le cœur du système. Il regroupe ce qui, dans la version initiale du cahier des charges, était séparé entre « Alerte » et « Incident ». Une alerte progresse à travers quatre statuts :

Statut	Signification	Déclenché par
NEW (Nouvelle)	L'alerte vient d'être créée, appel tout juste reçu	Création du formulaire « Nouvelle Alerte »
QUALIFIED (Qualifiée)	L'opérateur a analysé la situation et confirmé sa nature	Action « <code>change_status</code> »
TRANSMITTED (Transmise)	L'alerte a été transmise à un partenaire externe pour intervention	Action « <code>transmit</code> »
CLOSED (Clôturée)	L'événement est terminé	Action « <code>change_status</code> »

Chaque changement de statut est enregistré dans un historique (`AlertHistory`), conservant qui a fait le changement, quand, et un commentaire optionnel. Cet historique alimente aussi automatiquement le Journal des événements, sans code supplémentaire à écrire — un mécanisme Django appelé « `signal` » écoute chaque enregistrement en base et crée la ligne de journal correspondante.

Le numéro d'alerte

Chaque alerte reçoit un numéro unique et lisible, généré automatiquement côté serveur au format `ALT - [CODE_CENTRE] - [ANNÉE] - [NUMÉRO]`, par exemple `ALT - MRCC - ABJ - 2026 - 0001`. Ce numéro n'est jamais construit côté frontend, précisément pour éviter tout risque de doublon ou de désynchronisation entre deux opérateurs qui créeraient une alerte au même moment.

La transmission à un partenaire

Le centre ne dispose pas de flotte d'intervention propre : son rôle est de qualifier l'alerte puis de la transmettre à l'entité compétente (Marine Nationale, CIAPOL, Clinique Farah...). L'action `transmit` enregistre le partenaire notifié et l'horodatage exact de cette transmission — une exigence explicite du cahier des charges pour les cas de pollution, où l'heure de transmission au CIAPOL a une valeur légale et administrative.

7. Principaux points d'entrée de l'API

Méthode	URL	Description
POST	<code>/api/auth/token/</code>	Connexion : échange identifiant/mot de passe contre un token JWT
POST	<code>/api/auth/token/refresh/</code>	Renouvelle un token expiré sans redemander le mot de passe
GET	<code>/api/users/me/</code>	Profil du compte actuellement connecté
GET	<code>/api/references/...</code>	Récupère les listes déroulantes (catégories, priorités, sources...)
GET / POST	<code>/api/alerts/</code>	Liste des alertes / création d'une nouvelle alerte

Méthode	URL	Description
GET / PATCH	/api/alerts/{id}/	Détail d'une alerte / mise à jour
POST	/api/alerts/{id}/change_status/	Fait progresser l'alerte dans le workflow
POST	/api/alerts/{id}/transmit/	Transmet l'alerte à un partenaire externe
GET / POST	/api/sar/	Moyens de secours disponibles / création
GET / POST	/api/sar/engagements/	Engagement d'un moyen sur une alerte
GET / POST	/api/reports/daily/	Rapports de fin de journée
POST	/api/reports/daily/{id}/validate/	Valide le rapport et déclenche l'envoi email à la hiérarchie
GET	/api/reports/dashboard/	Indicateurs clés en temps réel (alertes, sauvetages, coordinations...)
GET / POST	/api/meetings/	Comptes-rendus de réunion avec pièce jointe PDF
GET	/api/event-logs/	Journal des événements (réservé aux comptes Admin / Super Admin)

Toutes les routes (sauf la connexion) exigent un token JWT valide, transmis dans l'en-tête HTTP Authorization: Bearer <token>.

8. Tests réalisés et validés

L'ensemble du parcours a été testé manuellement via Postman, de bout en bout, avec des résultats conformes aux attentes :

- Connexion et obtention d'un token JWT valide.
- Création d'une alerte avec génération automatique du numéro (ALT-MRCC-ABJ-2026-0001) et remplissage automatique du champ « créé par ».
- Changement de statut d'une alerte (NEW → QUALIFIED) avec apparition immédiate de l'entrée correspondante dans l'historique.
- Transmission d'une alerte à un partenaire (QUALIFIED → TRANSMITTED), avec horodatage exact enregistré.
- Vérification que le Journal des événements se remplit automatiquement à chaque action, sans intervention manuelle.
- Consultation du tableau de bord : agrégation correcte du nombre total d'alertes, du nombre de coordinations (alertes transmises), et de la répartition par catégorie et par statut.

Un correctif a été apporté en cours de route : le chargement optimisé des données liées (prefetch_related) capturait une version « figée » de l'historique avant qu'une nouvelle entrée n'y soit ajoutée dans la même requête. La solution a consisté à recharger explicitement l'objet juste avant de construire la réponse, garantissant que les données renvoyées sont toujours à jour.

9. Prochaines étapes recommandées

Sujet	Statut	Action recommandée
Carte de densité (heatmap)	Non commencé	Ajouter un endpoint renvoyant latitude/longitude de toutes les alertes pour alimenter une carte de chaleur côté frontend
Envoi d'email réel	Configuré en mode test (console)	Configurer un vrai serveur SMTP avant la mise en production
Champs prénom/nom des comptes	Vides pour l'instant	Les renseigner pour chaque compte afin d'afficher un nom lisible dans l'historique
Hébergement	En développement local (XAMPP)	Prévoir un serveur de production (~15 000 FCFA/mois) et une sauvegarde automatique
Intégration frontend	En attente	Le développeur frontend doit utiliser les identifiants (UUID) des référentiels, jamais leur libellé texte